PRACTICAL TASK

Spatial Relation Probing over Each Layer

The goal is to build a small synthetic dataset and benchmark whether different vision foundation models encode the spatial relation between two objects in their internal representations. To do this, we will:

1. Dataset design

First, we will design a synthetic dataset of 2D images containing two objects: a reference and a target. The four relations are **left**, **right**, **front**, and **back**. A 2D image has no real depth, so front/back can be simulated using size and occlusion:

- **Left / Right:** target is placed to the side of the reference, same size, no overlap.
- **Front:** target is larger (×1.5 of the base size), placed lower in the frame, and drawn on top of the reference (it occludes the reference).
- **Back:** target is smaller (×0.65 of the base size), placed higher in the frame, and drawn behind the reference (the reference occludes it).
- **Size jitter:** every object's size additionally varies by ±10% around its base size, for natural variation.
- **Reference position jitter:** the reference is not fixed at the exact center. Its position should be varied within a range of ±15% of the canvas size.

2. Scene generation

Each image contains one **reference** object and one **target** object, drawn with PIL (Python Imaging Library). Please follow these rules:

- Object shape must identify the role, not be shared between roles. Draw the reference only from {circle, square}, and the target only from {triangle, star, cross}. Without this split, there is no way to tell which object is “the reference” at all, and the relation label would not be recoverable from the image.
- Randomize, within each role's shape pool. Color (at least 6 colors), size (with some jitter, plus the front/back size rule above), and the reference object's position on the canvas.
- Add mild background variation (not the same flat color in every image).
- This randomization matters. Without it, a simple classifier could solve the task from the absolute pixel position instead of the actual relation between the two objects, and the result would not tell us anything about the model.
- Generate 250 images per relation (left, right, front, back) = 1000 images in total. (You can always generate more images for the sake of robustness if needed.)
- For each role, reserve exactly 2 of the 6 colors exclusively for the test split, so every shape in that role appears with 4 colors in training and the other 2 colors only at test time. This tests generalization to unseen appearances, not memorized ones.
- Keep both objects fully inside the canvas. After placing the reference and target, check both fit within a safety margin from the edges (e.g. 5–10% of the canvas size). If either would be clipped, resample its position or size for that image instead of letting it touch or cross the border. A clipped object is an even easier shortcut than size or occlusion.
- For left/right, ensure the horizontal distance between the reference and target centers exceeds the sum of their horizontal half-extents plus a small safety margin, so the two objects never touch. Independently, check that both objects remain fully inside the canvas (the clipping rule above). These are two separate checks, not one combined formula. Left/right must stay occlusion-free. Only front/back should ever overlap.
- Fix one random seed for the dataset generation itself (separate from the 3 seeds used later for probe training), so the dataset is exactly reproducible if you need to regenerate or share it.

• Before training anything, please generate and look at a handful of example images per relation (or plot the actual object offsets used) to catch bugs or overlapping classes early.

3. Models and representations

Please use exactly these three models:

Supervised ViT-B ImageNet pretrained	**Supervised ResNet-50** ImageNet pretrained	**Self-supervised ViT-B/16** DINO, ImageNet pretrained

NOTE 1: You are free to test other models if you would like to (but this is not required).

NOTE 2: You can use timm, torch hub, or torchvision models for any of these.

NOTE 3: Use each model's own official preprocessing (e.g. normalization).

For each ViT, extract and probe the following **separately**:

• The CLS token (skip if model doesn't have it — DINO and the supervised ViT-B have this).

• The mean-pooled patch tokens.

• Both of the above, from **every transformer layer**, not just the last one — this is called a layer sweep.

4. Probing protocol

• One **linear** probe per (model, layer, pooling type) combination — a 4-way classification problem (left / right / front / back).

• Split the dataset into 70/10/20% of images, balanced by relation. Training and validation draw only from the 4 training colors per role; test draws only from the 2 reserved colors per role (Section 2). Select the probe's regularization strength using the validation set, never the test set.

• Use three random seeds to create three different train/validation splits from the training-color image pool. Keep the same test set fixed across all three runs. Report mean ± standard deviation of test accuracy across the three runs. Because all three splits are drawn from the same training-color images, report this as split sensitivity, not as variability across independently generated datasets.

• In addition to the 4-way probe, train two separate binary probes: left-vs-right and front-vs-back, each on its own two classes only (not derived from the 4-way probe's outputs). Report their accuracies separately. Front/back can be solved from size and occlusion alone, without any real spatial understanding, so a combined 4-way number can hide a model that is strong on front/back for the wrong reasons while weak on left/right.

• Specify the exact extraction point for each layer's representation (e.g. each transformer block's output, i.e. the residual stream after that block). Use the same convention at every layer, and document any additional normalization applied before probing. Many ViTs use pre-normalization (LayerNorm before attention/MLP, not after), so don't assume a “final LayerNorm per block” exists.

5. Deliverable

• Before running any experiments, please write down at least two hypotheses about what you expect the probing results to show.

• Your code (dataset generator, feature extraction, and probing).

• A presentation of your results (e.g. the probe-accuracy vs. layer-depth plot, one line per model / pooling type). We will meet again, and you will present your findings to me.

• A short report (2 pages is enough) covering: Does the CLS token underperform the mean-pooled patch tokens? At which depth does accuracy peak? Does this differ between the supervised and the self-supervised ViT? How do your actual results compare to the two hypotheses you wrote down beforehand, and what (if anything) was unexpected?

• A short **Limitations** paragraph in your report: for example, this dataset tests whether spatial-relation information is *accessible* in a representation via a linear probe. High accuracy shows the information is present and linearly decodable. It does not by itself demonstrate

general spatial or 3D understanding.

6. Optional (bonus — not required to pass)

- **Cue isolation:** front/back uses three signals at once: size, vertical position, and occlusion. To find out which one the model actually relies on, build a few small extra versions of the dataset where only one signal changes at a time (the other two stay the same for front and back). Test the model on each version separately to see which single cue drives the accuracy.
- **Extra generalization check:** compare your validation accuracy (familiar, training colors) with your test accuracy (unseen, held-out colors). If they are close, the model is likely using the actual spatial relation. If validation is much higher than test, the model may be partly relying on recognizing familiar colors or shapes rather than the relation itself.
- **AbMILP patch aggregation:** as a third pooling type beyond CLS and mean-pooling, implement attention-based MIL pooling (AbMILP) — a small trainable module that learns a per-patch attention weight and takes the weighted sum of patch tokens, instead of a plain average. Reference: Przewięźlikowski et al. (arXiv:2412.03215); code: github.com/gmum/beyond_cls/blob/main/abmilp.py. AbMILP has its own trainable parameters (the attention-predictor MLP), so it is not a pure linear probe on frozen features. If it outperforms CLS/mean-pooling, that could reflect genuinely more informative patch-wise aggregation, or just added trainable capacity. Report its parameter count alongside its accuracy, and discuss this distinction rather than treating a win at face value.

Submission

Format

Please share your code (e.g. a GitHub repository or a zip file) together with your 2-page report ahead of our meeting, where you will present your results.

Timeline

We will agree on a deadline together after our meeting.